



Ministry of Housing and Urban Affairs
Government of India

Final Internship Report (HFA Division)

PREPARED BY

Satyam Gupta 218170942

Pranav Susana 210739

Vedant Kale 211153

Shamit Kamble 220999

Project # 1

Integrated Server-Side Automatic Document Compressor Tool

Problem Statement

Domain/Industry Context:

India's Pradhan Mantri Awas Yojana (PMAY-Urban) is focused on providing secure housing for the urban poor. The PMAY-Urban application form for a potential beneficiary requires them to upload documents such as Income Certificate, Land papers and Loan Sanction Letters in PDF format. These scans can often exceed the file size limits of the server.

Traditional Methods:

Users are forced to rely on third-party online PDF compression tools to reduce file size.

Limitations of Existing Methods:

- **Security/Privacy concerns:** User data like income info and loan information is exposed to unknown third-party websites.
- **Human dependency:** Users must manually compress and re-upload files.

Root Cause Analysis:

No secure, built-in mechanism is available on portals to handle large PDF uploads, pushing users to unsafe workarounds.

Proposed Solution

Idea Summary:

Although not AI-based, the solution is a critical enabler for AI-based platforms by ensuring secure and efficient document handling. The tool automatically compresses oversized PDFs at the backend before upload.

Key Features:

- Automated trigger on file upload
- Local compression using PyMuPDF or Ghostscript
- No data leaves the server — fully secure
- Transparent user notification post-compression

Product Overview:

- Users upload documents
- System checks size
- If oversized → compression is applied
- Compressed file is then accepted securely

Impact and Success Metrics

Key Goals:

- Eliminate data leakage risks
- Improve upload success rate
- Automate manual compression tasks

Quantifiable Metrics:

- 100% reduction in data exposure to 3rd parties
- 50–80% average reduction in file size
- Up to 30% improvement in submission completion rates

Detailed Product Design & Workflow

Modules:

- Upload Handler
- File Size Evaluator
- PDF Compression Engine
- Notification System

User Journey:

Upload → Compress (if needed) → Save → Confirm upload

Deployment Design:

- Backend in Python
- Local/on-premise server or secure cloud deployment
- Integrates with frontend file upload module

Product Development Plan

Data:

- Input: PDFs (Income, Land papers, Loan letters)
- Source: User uploads

Software Tools:

- Python
- Libraries: `fitz` (PyMuPDF), `Ghostscript`, `pdf2image`, `os`

Execution & Integration Strategy

- Backend module integrated into document upload form
- Minimal training required for admins
- Users see no change except improved upload experience
- Stakeholders: Web developers, backend integrators
- Low-cost open-source solution with minimal maintenance

Potential Challenges and Risks

- Image-heavy documents may lose quality if overly compressed
- Some rare PDFs may have encryption or format errors

Future Scope

- Integrate with AI-based OCR to validate compressed documents
- Offer user-side compression option as fallback

References

- https://pmaymis.gov.in/PMAYMIS2_2024/PmayDefault.aspx

- PyMuPDF (<https://pymupdf.readthedocs.io>)
- Ghostscript (<https://www.ghostscript.com/>)
- pdf2image library

Project # 2

AI-powered Automated Document Validation System for Secure Beneficiary Verification

Domain/Industry Context:

The PMAY-Urban application form for a potential beneficiary requires them to upload documents such as Income Certificate, Land papers and Loan Sanction Letters in PDF format. The validation of documents is a critical process in such domains where large volumes of beneficiary documents are processed for approvals (e.g., loan applications, welfare distribution).

Traditional Methods:

Currently, document validation is performed either manually or using basic rule-based checks. Often, documents are compressed or transferred to third-party websites for OCR (Optical Character Recognition), exposing sensitive data to security risks.

Limitations of Existing Methods:

- **Accuracy:**
Manual or rule-based checks are prone to human errors and inconsistent validation outcomes.
- **Efficiency:**
Manual validation is slow and labor-intensive, leading to delays in the process.
- **Scalability:**
Manual efforts are not easily scalable with growing document volume.
- **Human dependency:**
Heavy reliance on human verification increases bias and limits speed.
- **Security/Privacy concerns:**
Using third-party compression/OCR services can result in leakage of confidential beneficiary data.

Root Cause Analysis:

Current methods underperform mainly due to reliance on third-party tools for processing (risking data privacy), manual verification (introducing inefficiency and errors), and lack of integrated, automated validation scoring.

3. Proposed AI Solution

Idea Summary:

Develop an internal, automated document validation engine using open-source OCR and NLP models. This solution processes documents on secure servers, extracts and validates key information, scores document authenticity, and returns essential metadata (e.g., loan amount, timestamp) in a structured format.

Key Features:

- Automated Extraction: Detects and extracts relevant fields using OCR and NLP.
- Validation Scoring: Assigns confidence/validation scores to each field and document.
- Privacy Preserving: Processes everything on secure, internal servers, eliminating third-party risks.
- Structured Output: Returns a JSON with validated fields and confidence scores.

Product Overview:

- User Uploads Document
- Check File Validity (PDF or Image, size limits, format)
- Extract Text (Use PDF parsing; fallback to OCR if needed)
- Detect Document Type (Check for keywords like "Income Certificate", "Annual Income")
- Extract Key Fields (Beneficiary Name, Father's Name, Annual Income, Financial Year, Issue Date)
- Validate Field Values & Assign Confidence Scores
- Generate JSON Response with Extracted Data + Scores
- Return Results to User or Backend System

4. Impact and Success Metrics**Key Goals:**

- Enhance speed and accuracy of document verification.
- Reduce operational costs and manual effort.
- Safeguard beneficiary privacy by eliminating third-party data transfer.
- Automate and scale validation for high document volumes.

Quantifiable Metrics:

- 90%+ reduction in manual validation effort
- ~80% reduction in errors from manual processing
- 50-70% faster turnaround per document
- Zero third-party data exposure incidents

5. Detailed Product Design & Workflow

Modules:

- Document Upload Module: User-facing interface for document submission.
- OCR Module: Uses open-source models (e.g., Tesseract, EasyOCR) for text extraction.
- NLP & Validation Module: Extracts fields like loan value, time, beneficiary name; checks consistency, format, and authenticity.
- Scoring Engine: Assigns validation scores indicating confidence in correctness.
- Result Packaging Module: Compiles results into a structured JSON.

Model Pipeline:

- Type: CV (OCR) for text extraction; NLP for information parsing and validation.
- Output: JSON object with key-value pairs and validation scores.

Deployment Design:

- Stack: Python-based backend with REST API.
- Frontend: Web dashboard for upload/status.
- Deployment: On-premise or secure cloud (private VPC) for data privacy.

6. Product Development Plan

Data

- **Data requirements:**
Digital PDF scans of beneficiary documents, especially income certificate, in various quality/resolution. Each sample should have ground-truth labels (e.g., whether it's actually an income certificate, extracted name, etc.).
- **Data acquisition plan/pipeline:**
 - a. Collect a representative sample from historical applications and generate test cases with typical errors (e.g., skewed, noisy, low-contrast images).
 - b. New incoming documents from the web application are temporarily stored and processed using secure temp files, as per `tempfile.NamedTemporaryFile` in the code.
 - c. For each new document, a log of OCR outputs should be built to incrementally improve the validation pipeline.
- **Cleaning, labeling, feature engineering:**
 - i. Ensure documents are clear, legible, and in supported PDF format.
 - ii. Extract and maintain labels for document type (e.g., "income certificate" or "unknown"; mapped by `detect_document_type`), key values (name, UID, birth year, etc.) for downstream validation.
 - iii. (Future scope) Augment with synthetic documents to cover rare variants.

AI/ML Models

- **ML/DL techniques considered:**
 - a. OCR:

Uses [pytesseract](#) (Python bindings for the open-source Tesseract OCR engine). This extracts raw text from both native PDF text and scanned images (the code first checks for selectable PDF text via PyMuPDF, then falls back to OCR if needed).
 - b. Document Type Classification (rule-based):

The `detect_document_type` function checks whether the document contains income certificate or land document keywords after lowercasing all extracted text. This is a rule-based (not ML) approach but can be upgraded to a machine learning classifier if needed.
 - c. [Optional future] NLP parsing:

Further field extraction (e.g., finding the Aadhaar number, name, etc.) can use NLP libraries such as spaCy or a regex-based parser.
- **Tools/frameworks:**
 - a. pytesseract (Python OCR)
 - b. PyMuPDF (fitz) for PDF parsing
 - c. pdf2image (converts PDF pages to images for OCR)
 - d. Python for core logic
 - e. Flask for web backend

Software Tools

- **Programming languages:**
 - a. Python (all processing code)
- **Frontend/backend frameworks:**
 - a. Flask web server (handles file upload and response)
 - b. (Optional for UI) HTML forms
- **DBMS and other tech stack:**
 - a. No DBMS currently but can be extended with PostgreSQL or MongoDB if storing result logs and metadata.

Hardware Requirements / Development

- **Inference:**

The code is designed to run on a standard on-premise or cloud virtual server (recommended with at least 2 CPU cores and 4+ GB RAM for reasonable throughput).
- **No GPU required for Tesseract or the current workflow.**
- **Deployment:**

- a. Should run on internal secure infrastructure to ensure documents are not exposed externally.
- b. Can be containerized (Docker) for easier deployment.

7. Execution & Integration Strategy

- Implementation Plan: Phased rollout; begin with limited document types, expand based on feedback.
- Training: Onboarding guides for staff, in-app tooltips.
- Publicizing/Adoption: Internal workshops, pilot programs.
- Stakeholders: IT, compliance/legal, ops teams.
- Maintenance: Scheduled model/batch retraining, support for bug reporting.
- Cost: Includes hardware, licensing (for open-source, mostly minimal), training overhead.

8. Potential Challenges and Risks

- Data availability/quality: Poor scans or handwritten fields may reduce accuracy.
- Model biases: If training data is not diverse, NER performance may differ by document style/language.
- User acceptance: Staff may initially resist AI-based automation.
- Integration: Bridging with legacy systems may require custom connectors.
- Cost overruns: Due to unforeseen scaling requirements.

9. Future Scope

- Enhancements: Expand support for more document types, add language coverage, implement active learning feedback loops.
- Extensions: Apply to other domains—insurance, education, healthcare.
- Vision: Build a unified, automated document intelligence platform reducing human involvement to exception handling only.

10. References

Research Papers

- **Document Scanning and Check Valid Document Using AI**
Presents an AI-powered document validation system using Tesseract OCR, Flask, and large language models for extracting and verifying fields like name and date of birth.
<https://www.ijcrt.org/papers/IJCRT25A5469.pdf>
- https://pmaymis.gov.in/PMAYMIS2_2024/PmayDefault.aspx

Tools/Libraries Used

- **OCR Libraries**
 - a. **Tesseract OCR**: Widely used open-source OCR engine capable of recognizing over 100 languages; integrates with Python via pytesseract.
 - b. **PaddleOCR**: Modern, deep-learning-based OCR toolkit with flexible multilingual capabilities and deployment options.
 - c. **EasyOCR**: Open-source OCR library supporting 80+ languages, simple integration with Python.
 - d. **PyMuPDF (fitz)**: For extracting text and images directly from PDF files.

Existing Similar Solutions & Benchmarks

- **Google Cloud Document AI**
Provides high-accuracy, production-grade document processors. Benchmarked using precision, recall, F1, and field-specific metrics such as true/false positive extraction rates.
- **Microsoft Azure Document Intelligence**
Cloud-based document processing API with benchmarks reported on field extraction and document type recognition.
- **Veryfi Receipt Processing**
Benchmarks show deterministic AI models for receipt/document verification achieving over 99% accuracy on complex real-world formats.
- **Github solution**: https://github.com/shiva2410/Document_verification

Project #3:

AI-Based Clustering of Rooftop Materials Using Panchromatic Satellite Images

2. Problem Statement

Domain/Industry Context:

India's Pradhan Mantri Awas Yojana (PMAY-Urban) is focused on providing secure housing for the urban poor. Accurate classification of housing as Kutcha (temporary), semi-pucca, or Pucca (permanent) is essential for targeting upgrades and forming effective policies. Manual survey methods, which cover about 15 houses per person per day, are prohibitively slow and costly for the 10 million households required by PMAY-Urban.

Traditional Methods:

Housing type surveys rely on field teams physically visiting households, visually identifying rooftop material, and recording data manually—a process prone to delay, error, and limited scalability.

Limitations of Existing Methods:

- **Accuracy:** Visual inspections are subjective and error-prone.
- **Efficiency (time, cost):** Field surveys are time-consuming and resource-intensive.
- **Scalability:** Unable to cover millions of households within tight deadlines.
- **Human Dependency:** Success depends on training, consistency, and diligence of surveyors.
- **Security/Privacy Concerns:** Transport and aggregation of large volumes of paper or digital survey data can risk data integrity.

Root Cause Analysis:

Manual processes falter due to limited coverage speed, logistical challenges, subjective observations, and escalating costs with larger sample sizes, leading to delayed and potentially unreliable data for policy-making.

3. Proposed AI Solution

Idea Summary:

Implement an end-to-end AI system using high-resolution (25–30cm) panchromatic imagery from the Cartosat-3 satellite paired with deep learning models to automatically detect, segment, and classify urban rooftops by material type (Kutchra, semi-pucca, Pucca). All rooftops are geo-tagged and compiled into a cloud database for analytics and decision support.

Key Features:

- **Automated Rooftop Segmentation:** Deep learning models (e.g., U-Net, Mask R-CNN) detect and segment rooftop regions.
- **Hybrid Feature Extraction:** System extracts geometric, textural, and deep CNN features from each rooftop patch.
- **Unsupervised Clustering of Materials:** Advanced clustering algorithms group rooftop patches by material, minimizing need for labeled data.
- **End-to-End Scalability:** Batch processing of vast urban expanses with frequent updates as new imagery is acquired.
- **Web-Based Visualization:** Geo-tagged results accessible via interactive maps.
- **Audit Trail:** All key processing steps and outputs logged for transparency.

Product Overview (Workflow Sketch):

- Boundaries and areas of interest are uploaded.
- Panchromatic Cartosat-3 images are acquired and pre-processed.
- Rooftops segmented via deep learning models.
- Hybrid features are calculated for each rooftop.
- Clustering algorithms group roofs by material type.
- A city-scale database and map visualization is generated.

4. Impact and Success Metrics

Key Goals:

- Rapid, consistent, and objective mapping of rooftops by material for urban housing programs.
- Dramatic reduction in survey cost and time—scaling to millions of households.
- Provide actionable data to planners, energy researchers, and disaster managers.

Quantifiable Metrics:

- 90%+ reduction in time and labor over manual surveys.
- Over 95% segmentation accuracy (IoU) and high clustering quality (NMI, ARI) versus ground-truth samples.
- Up-to-date citywide maps updated monthly or seasonally.
- Scalable batch processing capable of mapping thousands of km² per day.

5. Detailed Product Design & Workflow

Modules:

- **Area & Image Acquisition:** Users specify boundaries; Cartosat-3 scenes are ordered and georeferenced.
- **Preprocessing:** Radiometric and geometric corrections applied for consistency.
- **Rooftop Detection:** Deep learning (U-Net, Mask R-CNN) segments and geo-tags rooftops.
- **Feature Extraction:**
 - a. *Geometric:* Area, shape, regularity.
 - b. *Textural:* GLCM, LBP.
 - c. *Deep Features:* CNN layer activations.
- **Clustering Pipeline:** Scalable K-Means and spectral clustering assign rooftops to Kutcha, semi-pucca, or Pucca clusters.
- **Database & Visualization:** Compiled, geo-tagged outputs published to a web-based GIS dashboard with analytics.

Model Pipeline:

- **Models Used:**
 - a. CNN-based segmentation (e.g., U-Net, Mask R-CNN).
 - b. Feature engineering (scikit-image, OpenCV, PyTorch).
 - c. Unsupervised clustering (scikit-learn).
- **Output Format & Flow:**
 - a. Polygon shapefile with rooftop type, cluster, and geographic coordinates.

Deployment Design:

- **Backend:** Python-based processing (TensorFlow/PyTorch, OpenCV, scikit-learn).
- **Frontend:** GIS web app for querying and visualization.
- **Deployment:** On-premise or secure cloud servers with scalable compute.

6. Product Development Plan

Data:

- **Requirements:** High-resolution (25–30cm) Cartosat-3 panchromatic images, city/ward shapefiles, expert-labeled sample rooftops for periodic benchmarking.
- **Acquisition Plan:**
 - a. Acquire satellite scenes per urban area from NRSC/ISRO partners.
 - b. Preprocess and align imagery with open GIS and remote sensing toolkits.
 - c. Regularly refresh imagery to maintain near-real-time maps.
- **Data Pipeline:**
 - a. Input shapefiles and images → preprocessing → segmentation → feature extraction → clustering → geo-tagged database.
- **Cleaning & Labeling:** Validate and denoise sample outputs through expert review and random field checks.

AI/ML Models:

- Deep learning (CNN-based) for segmentation, clustering algorithms for material grouping.
- Tools: TensorFlow, PyTorch, scikit-learn for algorithms; rasterio, OpenCV for preprocessing; QGIS for validation.

Software Tools:

- **Languages:** Python (primary), JavaScript (for frontend/map).
- **Frameworks:** TensorFlow, PyTorch, Flask/FastAPI, scikit-learn, QGIS, OpenCV.
- **DBMS:** PostGIS/PostgreSQL for spatial data; object storage for imagery.

Hardware Requirements/Development:

- Scalable CPU/GPU servers for batch processing.
- Integration-ready with municipal or national GIS backends.

7. Execution & Integration Strategy

- Pilot deployments in representative urban regions.
- Iterative rollout with field review and calibration.
- Integration with government portals/APIs for ongoing updates.

- Training for urban planners and government analysts.
- Publicity through official workshops and technical webinars.
- Stakeholder feedback loops and continuous system refinement.
- Annual budgeting for imagery, compute, and technical support.

8. Potential Challenges and Risks

- **Data Quality:** Cloud cover or poor image alignment can reduce accuracy.
- **Model Bias:** Urban heterogeneity requires periodic model retraining; expert validation ensures generalizability.
- **Scalability:** High data volumes require optimized hardware and software.
- **User Acceptance:** Continuous training and feedback collection to enhance trust in automated mapping.
- **Cost/Time Overruns:** Image acquisition and compute resource spikes can strain budgets if not carefully managed.

9. Future Scope

- **Multispectral & Multitemporal Analysis:** Integrate additional spectral bands for finer discrimination.
- **Expanded Geographies:** Cover rural and peri-urban areas.
- **Integration with Other Datasets:** Combine rooftop data with solar, infrastructure, or disaster risk analytics.
- **Advanced AI:** Adopt self-supervised and transfer learning methods.
- **Continuous Model Improvement:** Use real-world feedback and corrections for automated retraining.

10. References

- Indian Express – Satellite-based surveillance for asset mapping and housing.
- PMAY-Urban: 1.18 Crore Houses Sanctioned - Economic Survey.
- Cartosat-3 – Satellite specs and imaging details.
- Mapping Rooftop Materials Using High-Resolution Satellite Imagery and Convolutional Neural Networks.
- CNN-Based Detection of Building Roofs from High-Resolution Satellite Images.
- Major Advancements in Cartosat-3 - NRSC.
- Evaluation of Clustering Algorithms for Unsupervised Change Detection in VHR Remote Sensing Imagery.
- ScienceDirect – Unsupervised deep clustering for high-resolution satellite urban imagery.

Project # 4

Facial Matching System for Secure Beneficiary Verification During Geotagging

Domain/Industry Context:

Government welfare schemes, microfinance, and social benefit programs often require rigorous verification of beneficiaries. At geotagging points, it is essential to confirm that the individual physically present matches the records, preventing identity fraud and ensuring accountable distribution of entitlements.

Traditional Methods:

Historically, verification has relied on visual inspection by field personnel, manual comparison of photographs (e.g., on Aadhaar cards), and paperwork-based recordkeeping. In some cases, low-tech devices or basic image uploads are used for audits, lacking robust matching mechanisms.

Limitations of Existing Methods:

- **Accuracy:**
Subjective visual checks are highly error-prone, with significant risk of false acceptance or rejection.
- **Efficiency (time, cost):**
Manual inspection is slow, labor-intensive, and infeasible at scale or in remote settings.
- **Scalability:**
As schemes expand and volumes spike, human verification cannot keep pace, creating backlogs.
- **Human dependency:**
Relies on trained staff and their integrity; limited auditability and high chance of fatigue-driven mistakes.
- **Security/Privacy concerns:**
Paper trails and unsecured digital uploads increase the risk of data breaches and misuse of personal data.

Root Cause Analysis:

Current systems underperform due to reliance on subjective judgment, lack of automation, and insecure image handling. Absence of automated face matching and traceability allows identity fraud, inefficient distribution, and weak accountability.

3. Proposed AI Solution

Idea Summary:

Develop an automated facial matching system that compares the beneficiary's live-captured image at geotagging points with their Aadhaar card photograph. The system leverages deep learning-based facial recognition, robust pre-processing, and secure data handling to streamline, automate, and audit the verification process.

Key Features:

- **Automated Facial Matching:**
Deep learning models compare two facial images and assign a real-time match/no-match result.
- **Preprocessing Pipeline:**
Face detection, alignment, and normalization ensure images are comparable, regardless of source or device.
- **Threshold-based Decision:**
Adjustable match threshold, tuned to minimize both false positives and false negatives.
- **Audit Logging:**
Every match attempt, result, and exception is logged for traceability and compliance.

Product Overview

- **Beneficiary Photo Capture at Geotagging:**
The beneficiary's live facial image is captured at the scheme distribution site using a mobile device or camera.
 - **Aadhaar Image Acquisition:**
The corresponding Aadhaar card photo is obtained either from an uploaded image or a secure database, ensuring legal compliance.
 - **Face Detection & Alignment:**
Both the live and Aadhaar images are processed through face detection algorithms to isolate and standardize the facial region.
 - **Feature Extraction:**
Deep learning models transform both detected faces into compact feature vectors that represent their unique facial characteristics.
 - **Similarity Calculation:**
The system calculates a similarity score between the two face embeddings to assess if they likely belong to the same individual.
 - **Automatic Match or Flag for Manual Review:**
If the similarity exceeds a preset threshold, the system declares a 'Match'; otherwise, results are flagged for manual inspection if uncertain.
 - **Result Delivery & Audit Logging:**
The verification result and relevant metadata are conveyed to field operators and securely logged for traceability and audit purposes.
-

4. Impact and Success Metrics

Key Goals:

- Substantially reduce cases of identity fraud in welfare schemes.

- Automate verification to improve speed and lower manual workload.
- Ensure privacy and data security compliance.
- Enable transparent, scalable monitoring for all stakeholders.

Quantifiable Metrics:

- Match accuracy above 95% (as measured in field trials).
- 70–90% reduction in manual verification time per beneficiary.
- <2% rate of false accept/reject requiring manual review.
- 100% audit coverage of match attempts.
- Zero documented cases of data breach or unauthorized image access.

5. Detailed Product Design & Workflow

Modules:

- **Image Collection Module:**
Captures and uploads per-beneficiary photos at geotagging, accepts Aadhaar image input.
- **Preprocessing Module:**
Standardizes lighting, size, and alignment for both captured and document images.
- **Face Detection & Alignment:**
Locates facial region using algorithms (e.g., MTCNN, Haar Cascades), aligns for pose normalization.
- **Feature Extraction Module:**
Converts faces into mathematical vectors (embeddings) via deep CNNs (FaceNet, ArcFace, VGGFace) for robust comparison.
- **Matching & Decision Module:**
Computes similarity, applies threshold, outputs 'Match'/'No Match', and flags borderline cases for review.
- **Audit Logging Module:**
Logs each attempt with metadata for traceability.

Model Pipeline:

- **Type:**
Computer Vision (CV), Deep Learning (CNNs)
- **Output Format & Decision Flow:**
 - Inputs: Two images (live capture, Aadhaar)
 - Output: JSON with match decision, similarity score, and confidence level

Deployment Design:

- **Backend:**
Python, FastAPI/Flask server, models via PyTorch/TensorFlow.

- Frontend:
Web/mobile dashboard for field operators.
- Deployment:
On-premise servers or secure cloud (private VPC). API endpoints protected by authentication and encryption.

6. Product Development Plan

Data:

- Requirements:
Labeled pairs of facial images (live+document) with ground-truth match/non-match; sufficient variety in lighting, pose, age.
- Acquisition Plan:
Collect in-house with consent during real workflows; augment with public face datasets with similar demographics.
- Pipeline:
Secure upload → storage (encrypted) → preprocessing → manual/automated annotation → model training.
 - Cleaning: Remove out-of-focus or obstructed images.
 - Labeling: Human review for ambiguous or hard cases.

AI/ML Models:

- Techniques:
Convolutional Neural Networks for embedding (FaceNet, ArcFace, VGGFace); MTCNN or Haar cascades for face detection; cosine or Euclidean distance for matching.
- Tools/Frameworks:
PyTorch, TensorFlow (deep learning), OpenCV (image processing), Dlib (alignment), Python.

Software Tools:

- Programming Languages:
Python (backend/processing), JavaScript (optional for web/mobile frontend).
- Frameworks:
Flask or FastAPI for backend API, React Native or other web/mobile libraries for UI.
- DBMS:
PostgreSQL, MongoDB (results and logs), S3 (secure image storage).

Hardware Requirements/Development:

- Modern multi-core CPU for inference; GPUs recommended for large-scale/batch deployments or rapid prototyping.
- High-quality cameras on geotagging devices.

- Secure cloud or internal network infrastructure.

7. Execution & Integration Strategy

- Field deployment at select geotagging points for phased rollout.
- Onboarding and training for field personnel.
- User guides and support channels established.
- Stakeholder engagement (IT, scheme managers, legal/compliance).
- Publicity through program awareness drives.
- Maintenance plan with regular updates to models and security protocols.
- Cost allocation for devices, cloud compute, training, and support.

8. Potential Challenges and Risks

- Data quality:
Poor quality, obscured, or outdated images can hamper recognition.
- Model bias/fairness:
Risk of unfair performance across demographic groups if training data is unbalanced.
- Acceptance:
Field staff resistance or unreliable connectivity may hinder adoption.
- Integration complexity:
Compatibility issues with legacy systems.
- Cost/time overruns:
If large-scale photo collection or manual review is needed.

9. Future Scope

- Enhance models to better handle age variations and low-light conditions.
- Extend solution to other government and private sector ID verification.
- Incorporate liveness detection to prevent spoofing.
- Develop offline-capable, edge-processing workflow for remote use.
- Build analytics dashboard for program monitoring at scale.

10. References

Research Papers

- **"Two Proposed Models for Face Recognition: Achieving High Accuracy and Speed with Artificial Intelligence" (ETASR, 2024):** Presents traditional and deep learning models (Viola-Jones, LDA, 1D-CNN) specifically comparing performance accuracy for facial verification, relevant for scheme beneficiary use cases. Uses MUCT dataset for evaluation².
- **"End-to-End Protocols and Performance Metrics For Unconstrained Face Recognition" (ICB, 2019):** Examines benchmarking protocols, dataset selection, and real-world performance metrics, with references to major datasets like LFW and IJB series.

Tools and Libraries

- **OpenCV:** Comprehensive vision library with tools for face detection, alignment, and matching; used across industrial and academic projects.
- **Face Recognition (Python):** High-level wrapper using DLib and OpenCV; popular for rapid prototyping.
- **FaceNet:** Google's deep CNN producing face embeddings, pre-trained models publicly available and state-of-the-art for many benchmarks.
- **ArcFace, OpenFace, DeepFace, InsightFace:** Deep learning frameworks with high accuracy on verification and identification, used in research and commercial products.

Existing Solutions & Benchmarks

- **Amazon Rekognition:** Cloud-based real-time facial analysis and verification software, integral for several government pilots and enterprise projects.
- **Microsoft Face API:** Offers facial detection, recognition, and identity verification at scale; often used as an industry baseline.
- **Face++:** Commercial suite for face matching, ID verification, and access control; SDKs for integration with government solutions.
- **Cognitec FaceVACS:** Enterprise-grade solution with live video and document-to-photo matching, used in border control and large-scale identity programs¹².
- **NIST FRVT (Face Recognition Vendor Test):** Widely respected benchmark for commercial and open-source face recognition technologies; evaluates accuracy, speed, and robustness under real-world variability¹².
- **Clearview AI, FindFace, Luxand:** Additional solutions used by law enforcement and for public safety, leveraging large public datasets for recognition tasks¹².

Project #5:

Voice-Assisted Multilingual Form Filling System for Inclusive Beneficiary Services

2. Problem Statement

Domain/Industry Context:

Large-scale government and welfare schemes often require beneficiaries to complete digital forms. However, many users face literacy, language, or accessibility barriers, especially in a linguistically diverse country such as India. This limits the reach and efficiency of digital services intended for all citizens.

Traditional Methods:

Paper or digital forms are filled by hand or with clerical support, often leading to errors, delays, and the exclusion of the differently-abled or non-literate individuals. Translators or field staff are sometimes employed to assist, but this is costly and time-consuming.

Limitations of Existing Methods:

- Accuracy: Manual data entry is prone to errors and misinterpretation, particularly with multilingual users.
- Efficiency (time, cost): High operational costs for staff-based assistance; slower data collection.
- Scalability: Human support cannot be rapidly expanded to reach all corners of the country.
- Human Dependency: Relies on the presence and skill of assistants or translators.
- Security/Privacy Concerns: Sensitive data may be overheard or mishandled.

Root Cause Analysis:

Barriers to form completion arise from low literacy, language diversity (dialects, code-switching), disability, and limited digital literacy. Traditional processes cannot guarantee inclusion or scale to the national level.

3. Proposed AI Solution

Idea Summary:

Develop a voice-assisted form filling platform powered by automatic speech recognition (ASR), natural language processing (NLP), and voice feedback for real-time, accurate, and inclusive form completion in multiple Indian languages and dialects.

Key Features:

- Speech-to-Text Engine: Converts live speech into text in real-time with high accuracy.
- Multilingual Support: Handles a variety of Indian languages, dialects, accents, and code-switching scenarios.
- Form Field Mapping: Intelligently maps recognized speech to the correct form fields.
- Interactive Voice Guidance: Gives meaningful prompts, feedback, and clarification requests through TTS.

- Data Validation & Correction: Validates inputs and guides corrections via voice.
- Accessibility: Designed for users with limited literacy or physical ability.

Product Overview (Workflow Sketch):

- User is welcomed and guided by audio prompts.
- Each field is prompted and user responds by speaking.
- System transcribes and parses response, maps to form field.
- Data is validated instantly; clarification is requested if needed.
- Upon completion, form is read back for user confirmation before submission.

4. Impact and Success Metrics

Key Goals:

- Enable inclusive and accessible form completion for diverse populations.
- Speed up data collection with minimal manual support.
- Ensure data accuracy and lower error rates.
- Expand reach to non-literate and differently-abled beneficiaries.

Quantifiable Metrics:

- 85%+ reduction in manual form-filling assistance required.
- Support of 15+ Indian languages and key dialects.
- Field data accuracy rates above 95% after voice-based validation.
- Usability success rate above 90% for non-literate or elderly users in pilot studies.
- Feedback and form correction error rate below 2%.

5. Detailed Product Design & Workflow

Modules:

- Voice Interaction Module:
 - a. Initiates session, delivers prompts, confirms input via TTS (Text-to-Speech).
- ASR Engine:
 - a. Converts speech into text; supports multilingual and code-switching input.
- NLP & Entity Extraction:
 - a. Parses text, extracts relevant information for each form field.
- Form Logic Module:
 - a. Maps extracted values to correct form fields, triggers validation.
- Validation & Correction Module:
 - a. Checks format and plausibility; requests clarification as needed.
- Review & Confirmation:
 - a. Reads back transcribed and mapped entries for user verification.

- Audit Logging:
 - a. Logs all user interactions and field entries for traceability.

Model Pipeline:

- ASR Model:
 - a. Multilingual, accent-tolerant models (e.g., Whisper, IndicASR, Kaldi).
- NLP Model:
 - a. Language and region-specific NER, text parsing (spaCy, transformers).
- TTS Model:
 - a. Multilingual synthesizers for inclusive guidance (eSpeak, AI4Bharat TTS).
- Validation Engine:
 - a. Regex, rule-based, and semantic checks per field.

Deployment Design:

- Backend: Python (Flask/Django), Node.js for business logic, data storage (PostgreSQL/MongoDB).
- Frontend: React/Angular/Flutter web/mobile app for voice interaction.
- Deployment Environment: Secure on-premise or cloud infrastructure, with encryption for stored audio and data.

6. Product Development Plan

Data:

- Data Requirements:
 - a. Large corpus of labeled audio form responses in 15+ Indian languages and dialects.
 - b. Parallel textual and spoken datasets for model training.
- Acquisition Plan:
 - a. Use open datasets (IndicVoices, IndicVoices-R, CommonVoice, FLEURS, Bhashini).
 - b. Collect more audio through partnership with NGOs/government drives and open sourcing.
 - c. Annotate data (transcription, entity tagging) for model refinement.
- Data Pipeline:
 - a. Audio recording → annotation → model training → continuous improvement pipeline.
 - b. Synthetic data via TTS for underrepresented languages as stop-gap.

AI/ML Models:

- Speech-to-Text:
 - Whisper, Kaldi, IndicASR, wav2vec, SpeechBrain.
- NLP:
 - spaCy, Rasa NLU, multilingual transformers for field and value extraction.

- TTS:
 - eSpeak, Festival, AI4Bharat TTS for prompts and feedback.
- Validation:
 - Python tools (Cerberus, Marshmallow) for validation pipelines.

Software Tools:

- Programming Languages: Python, JavaScript.
- Frameworks: Flask, Django, React/Angular/Flutter.
- DBMS: PostgreSQL/MongoDB for logs and user data.

Hardware Requirements/Development:

- Conventional CPU servers; scalable cloud GPU capacity for ASR training and inference.
- Mobile devices or standard desktops for user interface deployment.

7. Execution & Integration Strategy

- Pilot Launches: Roll out in select districts with major Indian languages for usability feedback.
- Training & Support: Onboard field agents, provide digital training modules and manuals.
- Stakeholder Engagement: Partner with government departments, NGOs, accessibility groups.
- Iterative Development: Deploy feedback-driven updates; regular model retraining.
- Outreach: Awareness sessions, inclusion champions for marginalized communities.
- Maintenance: Dedicated team for updates, tech support, and bug fixing.
- Cost: Budget for cloud compute, audio dataset acquisition, ongoing improvement.

8. Potential Challenges and Risks

- Data Scarcity: Some dialects/accents may be poorly represented in training data.
- Recognition Bias: Possible accuracy drop for rare dialects or speech patterns.
- Connectivity Gaps: Real-time voice input may be hindered by poor network; consider offline-capable options.
- Privacy: Ensure all speech data is encrypted and handled per legal requirements.
- Adoption: Resistance from users unfamiliar with digital voice interfaces.
- Time/cost overruns: ASR/NLP development is resource-intensive; iterative tuning is required.

9. Future Scope

- Expand to all 22+ official Indian languages and major dialects.
- Incorporate support for voice-enabled authentication and digital signatures.
- Improve continuous learning by passively updating models with field-collected data.
- Add advanced accessibility features for visually impaired or differently-abled users.
- Integrate with form analytics dashboard to monitor progress and bottlenecks.

10. References

- DigiForm AI-based form filling
- AI4Bharat IndicVoices-R dataset
- IndicVoices dataset — Papers with Code
- Hybrid ASR architectures (ICON 2021)
- Multilingual and Code-Switching ASR Challenge
- Automated Voice Based Form Filling (GitHub)
- Multilingual Speech Recognition with a Single End-to-End Model

Gallery



